

Informe Prueba 2

Asignatura: Infraestructura para Ciencia de Datos

Profesor:. Michael Miranda

Autor: Fabrizio Larco Jorquera

Fecha: 08 de Julio de 2026

1. Contexto Conceptual y El Caso de Uso

El Caso de Estudio: Auditoría SA-MP

Este proyecto desarrolla una infraestructura para auditar los registros (*logs*) de un servidor de San Andreas Multiplayer (SA-MP). En estos servidores se generan miles de eventos diarios de transacciones económicas y apertura de *lootboxes*. El sistema analiza estos registros masivos en busca de fraudes (ej. transferencias anómalas mayores a \$5,000,000 o alteración de probabilidades de ítems legendarios). Debido al volumen de datos, la tarea de detección debe ser delegada a un proceso en segundo plano, y sus resultados deben ser sellados criptográficamente para evitar que administradores corruptos borren la evidencia material.

2. Arquitectura Implementada (Ejercicios 2 y 3)

2.1. Ambiente Reproducible (Docker)

El ecosistema se encapsuló utilizando Docker y Docker Compose, empleando la imagen `python:3.10-slim`. Esto garantiza que la instalación de dependencias nativas y las rutas de los volúmenes compartidos funcionen idénticamente en cualquier entorno, resolviendo el problema de inconsistencia entre entornos de desarrollo y producción.

2.2. Orquestador Asíncrono

- **Frontend (Streamlit):** Panel de control interactivo que ingiere los archivos `.log`, genera un ID de tarea y encola el trabajo. Utiliza un patrón de *Polling* acotado para monitorear el progreso dinámicamente sin congelar la pestaña del navegador.
- **Broker (Redis):** Actúa como memoria compartida. Almacena los trabajos en una cola (`training_queue`) operando bajo el principio FIFO y gestiona el estado del procesamiento.
- **Worker (Python Backend):** Proceso aislado y sin interfaz gráfica que escucha a Redis. Al recibir la tarea, ejecuta el análisis de los logs mediante expresiones regulares, aísla los fraudes y guarda las anomalías en la carpeta física de resultados.

3. Registro Verificable: Implementación de Blockchain (Ejercicio 4)

Para garantizar la inmutabilidad de los reportes generados, se desarrolló una Blockchain didáctica, asegurando que ninguna auditoría pueda ser alterada *a posteriori* sin ser detectada.

3.1. Estructura Criptográfica y Secuencialidad

El historial de orquestación se empaqueta en fragmentos cronológicos.

- El **Bloque 1** contiene el primer lote de eventos procesados y su propio código hash SHA-256.
- El **Bloque 2** almacena el siguiente lote de eventos, pero guarda celosamente la firma original del Bloque 1 en su variable de enlace (`previous_hash`), creando una dependencia estricta.

3.2. Árbol de Merkle

Para optimizar la estructura del bloque, se implementó una función constructora de Árboles de Merkle. Toma todos los eventos de auditoría de un lote específico, los empareja y calcula hashes recursivos hasta obtener una única **Raíz de Merkle** (*Merkle Root*). Cualquier alteración mínima en un log modifica esta raíz por completo.

3.3. Benchmark de Prueba de Trabajo (PoW)

Se configuró un algoritmo determinista para minar los bloques exigiendo un prefijo de ceros. Los resultados obtenidos empíricamente en la plataforma demuestran el aumento del costo computacional (Efecto de Dificultad):

Dificultad	Intentos (Nonce)	Tiempo de Ejecución (s)	Hash Resultante (Muestra parcial)
1	38	0.0003	086a8d6d188151...
2	129	0.0007	00ad1e79773365...
3	5,969	0.0322	000b2b177baf17...

3.4. Efecto Dominó y Simulacro de Ataque

Para probar la robustez del sistema, se programó un escenario de ataque donde se inyecta intencionalmente un evento falso en un bloque ya sellado. El sistema de validación criptográfica reacciona de la siguiente manera:

1. Debido al **Efecto Avalancha** del algoritmo SHA-256, el hash propio del Bloque 1 muta drásticamente al recibir el dato fraudulento.
2. Al evaluar la cadena, el validador contrasta este nuevo hash con la firma original que el Bloque 2 mantiene en su memoria inmutable.
3. Al no existir coincidencia, el puntero se rompe, deteniendo la evaluación y arrojando una alerta crítica de inconsistencia. El fraude queda matemáticamente expuesto.

4. Comandos de Ejecución

Para levantar la infraestructura y ejecutar las pruebas, se utilizan los siguientes comandos en la terminal de Linux:

1. **Levantamiento de contenedores en segundo plano:**
`sudo docker compose -f docker/docker-compose.yml up --build -d`
2. **Inyección de datos masivos y anomalías:**
`python data/generadorLogs.py`
3. **Despliegue del panel integral (Interfaz Asíncrona + Blockchain):**
`streamlit run appAsync/app.py`

5. Limitaciones y Conclusión

5.1. Cuándo NO usar esta arquitectura

La arquitectura de colas y contenedores introduce sobrecarga por latencia de red interna, procesos de serialización de datos y consumo de memoria del broker. **No se recomienda** para procesamiento trivial que demoran menos de un segundo, sistemas transaccionales críticos de ultra-baja latencia (ej. *High-Frequency Trading*), o etapas exploratorias tempranas donde el costo operativo supera la conveniencia interactiva.

5.2. Limitaciones de la Blockchain Didáctica

Aunque este ejercicio demuestra exitosamente la inmutabilidad de los datos locales, representa un sistema centralizado. En un escenario productivo real, este modelo fallaría si el servidor físico central es vulnerado a nivel de disco. Una infraestructura Blockchain corporativa requiere mecanismos de consenso distribuido entre múltiples nodos independientes, criptografía de llave asimétrica para validación de autoría y tolerancia a particiones de red.